



DEPI R4 - Data Engineering Track Group:

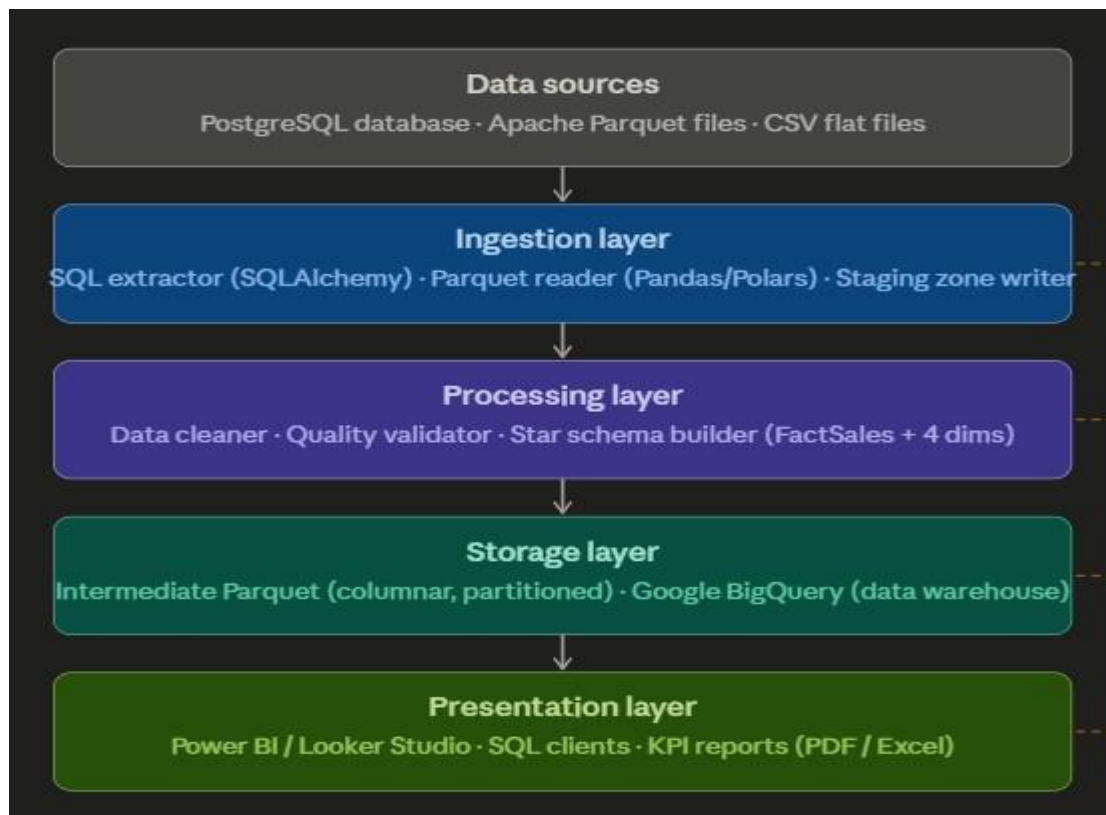
R4_DEPI4_GIZ4_AIS5_S3

Project 5: Building Retail Data Pipeline

➤ Architectural Style

#	Layer	Description	Technologies
1	Data Source Layer	Raw data origins – SQL relational databases and Parquet flat files containing Walmart-style retail transaction data.	PostgreSQL, Apache Parquet files
2	Ingestion Layer	Extracts raw data from source systems and writes it unchanged to the staging zone. Handles both SQL queries and file reads.	Python (Pandas / Polars), Airflow DAG
3	Processing Layer	Cleans, validates, and transforms raw data: deduplication, null handling, type correction, format standardization, and Star Schema construction.	Python (Pandas / Polars), Great Expectations
4	Storage Layer	Hosts the cleaned / transformed data in columnar format for intermediate storage (Parquet) and final analytical querying (Data Warehouse).	Apache Parquet, Google Big Query
5	Presentation Layer	Exposes the processed data to BI tools and reporting systems for dashboards, KPI reports, and SQL-based analysis.	Power BI / Looker Studio, SQL Clients

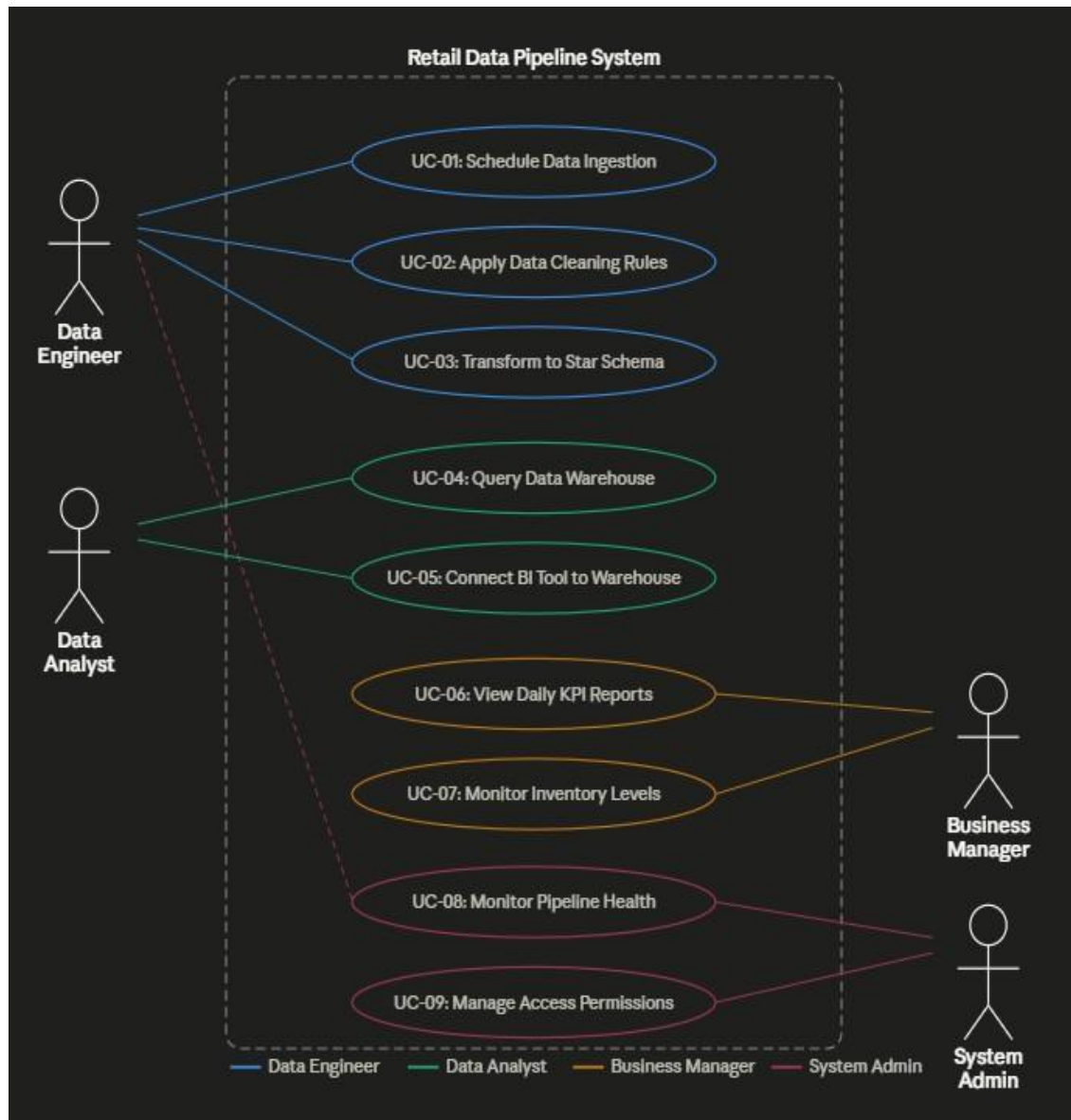
➤ Architecture Diagram



➤ System Actors

Actor	Type	Role	Key Responsibilities
Data Engineer	Primary Actor	Pipeline architect & builder	Designs ingestion, writes cleaning scripts, manages Airflow DAGs
Data Analyst	Secondary Actor	Primary data consumer	Builds dashboards, runs SQL queries on the Data Warehouse
Business Manager	Secondary Actor	Decision-maker	Reviews KPI reports, monitors inventory, drives strategic decisions
System Administrator	Supporting Actor	Infrastructure support	Manages Docker/cloud environment, enforces security & uptime

➤ Use Case Diagram



➤ Descriptions

- 1) **UC-01:** The Data Engineer schedules a daily automated pipeline that extracts data from SQL databases and Parquet files into the staging zone via Apache Airflow, without manual intervention.
- 2) **UC-02:** The Data Engineer applies cleaning rules to remove duplicates, fix nulls, correct data types, and standardize formats before data enters the warehouse.
- 3) **UC-03:** The Data Engineer transforms cleaned data into a Star Schema with a FactSales table and four-dimension tables (Date, Product, Customer, Supplier).
- 4) **UC-04:** The Data Analyst queries about the Data Warehouse using SQL to analyze sales trends and generate business insights within a 10-second response SLA.
- 5) **UC-05:** The Data Analyst connects a BI tool (e.g., Power BI) to the Data Warehouse via JDBC/ODBC to build interactive dashboards filtered by date, product, and region.
- 6) **UC-06:** The Business Manager views daily KPI reports (total revenue, units sold, top products) and exports them as PDF or Excel for strategic decision-making.
- 7) **UC-07:** The Business Manager monitors real-time and historical inventory levels per product and location, with low stock alerts and 12-month trend visibility.
- 8) **UC-08:** The System Administrator monitors all Airflow DAG statuses in real-time, receives failure alerts within 5 minutes, and reviews logs retained for 30 days.
- 9) **UC-09:** The System Administrator manages RBAC across all system components, ensuring role-appropriate data access, with all permission changes audit-logged.

➤ Functional

ID	Requirement	Description
FR-01	Automated ingestion	Extract data from SQL and Parquet files daily via Airflow.
FR-02	Data staging	Store raw ingested data in a staging zone before transformation.
FR-03	Data cleansing	Remove duplicates, fix nulls, correct types, and standardize formats.
FR-04	Star schema transform	Transform cleaned data into FactSales + 4 dimension tables with computed metrics.
FR-05	Orchestration	Use Airflow to schedule, retry, log, and alert on all pipeline tasks.
FR-06	Warehouse loading	Load transformed data into BigQuery for SQL querying.
FR-07	Quality validation	Validate uniqueness, completeness, and referential integrity at each stage.
FR-08	BI integration	Expose the warehouse via JDBC/ODBC for Power BI dashboards.
FR-09	KPI reporting	Daily KPI reports (revenue, units, top products) exportable as PDF/Excel.
FR-10	Inventory monitoring	Inventory levels per product/location with low-stock alerts and 12-month trends.
FR-11	Pipeline monitoring	Real-time DAG status and failure alerts within 5 minutes.
FR-12	RBAC	Analysts access only processed data; all permission changes are logged.

➤ Non-Functional

ID	Category	Requirement	Target
NFR-01	Performance	SQL query response time ≤ 10 seconds.	≤ 10 sec / query
NFR-02	Performance	Data refreshes daily by 6:00 AM.	Daily by 06:00 AM
NFR-03	Performance	Failure alerts delivered within 5 minutes.	≤ 5 min latency
NFR-04	Scalability	Handle growing volumes via Parquet + BigQuery elastic compute.	Partitioned Parquet
NFR-05	Scalability	Support new sources or streaming without pipeline redesign.	Modular design
NFR-06	Reliability	$\geq 99\%$ successful daily runs over a 30-day window.	$\geq 99\%$ success rate
NFR-07	Reliability	Auto-retry failed tasks at least 3 times before alerting.	Min. 3 retries
NFR-08	Availability	Data Warehouse accessible 24/7.	$\geq 99.5\%$ uptime
NFR-09	Security	RBAC enforced; analysts blocked from raw staging tables.	Least-privilege
NFR-10	Security	Log all access changes and unauthorized attempts.	Audit trail ≥ 30 days
NFR-11	Security	Encrypt all data at rest and in transit.	AES-256 / TLS 1.2+
NFR-12	Maintainability	Containerize pipeline using Docker Compose.	Single-command deploy
NFR-13	Maintainability	Retain all pipeline run logs for at least 30 days.	≥ 30 days retention
NFR-14	Maintainability	Each pipeline stage independently modifiable.	Decoupled components
NFR-15	Usability	Airflow dashboard shows DAG status without code inspection.	Graphical UI
NFR-16	Usability	BI dashboards support filters without SQL knowledge.	Filter-based UI
NFR-17	Compatibility	BI tool supports JDBC and ODBC protocols.	JDBC / ODBC
NFR-18	Recoverability	Re-run only failed tasks (idempotent execution).	Partial re-run

Data Flow & System Behavior – ERD & Mapping

Entity Relationship Diagram (ERD)

ERD: Retail Data Model with Cardinality & Participation

The ERD represents the conceptual structure of the retail system, showing entities, attributes, and relationships along with their cardinality and participation constraints.

Entities & Attributes:

Customer: customer_id (PK), first_name, last_name, email, phone, address, join_date

Order: order_id (PK), order_date, total_amount

OrderItem (Weak Entity): orderitem_id (partial key), quantity, unit_price, discount

Product: product_id (PK), product_name, price, stock_qty

Category: category_id (PK), category_name, description

Supplier: supplier_id (PK), supplier_name, contact_name, phone, email

Store: store_id (PK), store_name, location, phone

Relationships:

Customer places Order (1:N)

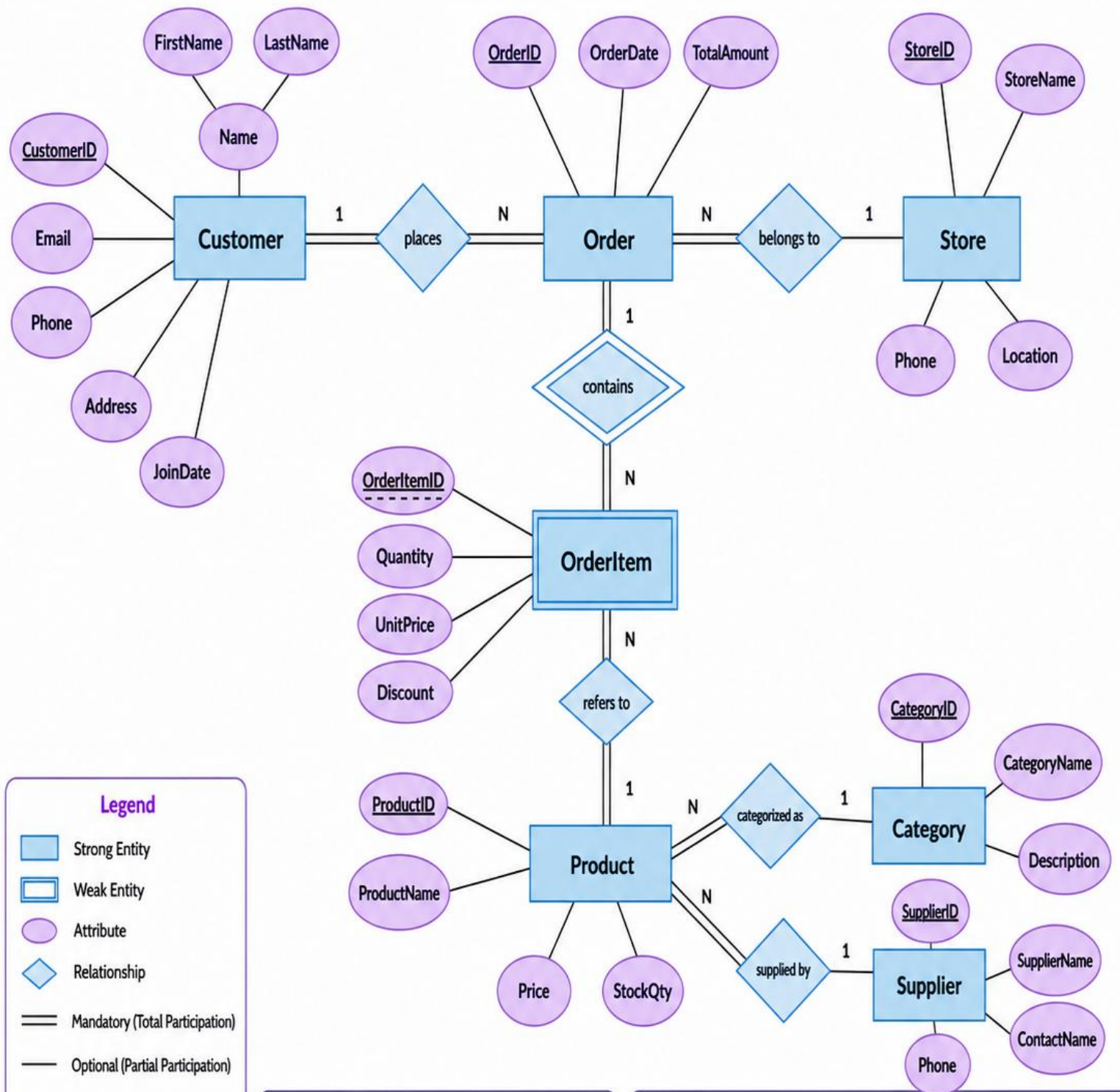
Order belongs to Store (N:1)

Order contains OrderItem (1:N, identifying relationship)

OrderItem refers to Product (N:1)

Product categorized as Category (N:1)

Product supplied by Supplier (N:1)



Cardinality & Participation Summary

Relationship	Cardinality	Participation	
		From Left Entity	From Right Entity
Customer — places — Order	1 : N	Customer (1,N)	Order (1,1)
Order — belongs to — Store	N : 1	Order (1,1)	Store (0,N)
Order — contains — OrderItem	1 : N	Order (0,N)	OrderItem (1,1)
OrderItem — refers to — Product	N : 1	OrderItem (1,1)	Product (0,N)
Product — categorized as — Category	N : 1	Product (1,1)	Category (0,N)
Product — supplied by — Supplier	N : 1	Product (1,1)	Supplier (0,N)

Business Rules (Realistic)

- ✓ An Order is placed by a Customer, but a Customer may place zero or more Orders.
- ✓ Each Order must belong to one Store, while a Store may have zero or more Orders.
- ✓ An Order may contain one or more OrderItems, but an Order may exist without items (i.e., draft order).
- ✓ Each OrderItem must refer to one Product, but a Product may not be referenced in any OrderItem yet.
- ✓ Each OrderItem stores the Quantity, UnitPrice, and Discount for the product at the time of the order.
- ✓ Each Product must belong to one Category, while a Category may have zero or more Products.
- ✓ Each Product must be supplied by one Supplier, while a Supplier may supply zero or more Products.
- ✓ Category and Supplier may exist even if not used yet.

Relational Schema Mapping

Relational Schema derived from ERD

This section converts the ERD into relational tables with primary keys and foreign keys.

Customer(customer_id PK, first_name, last_name, email, phone, address, join_date)

Store(store_id PK, store_name, location, phone)

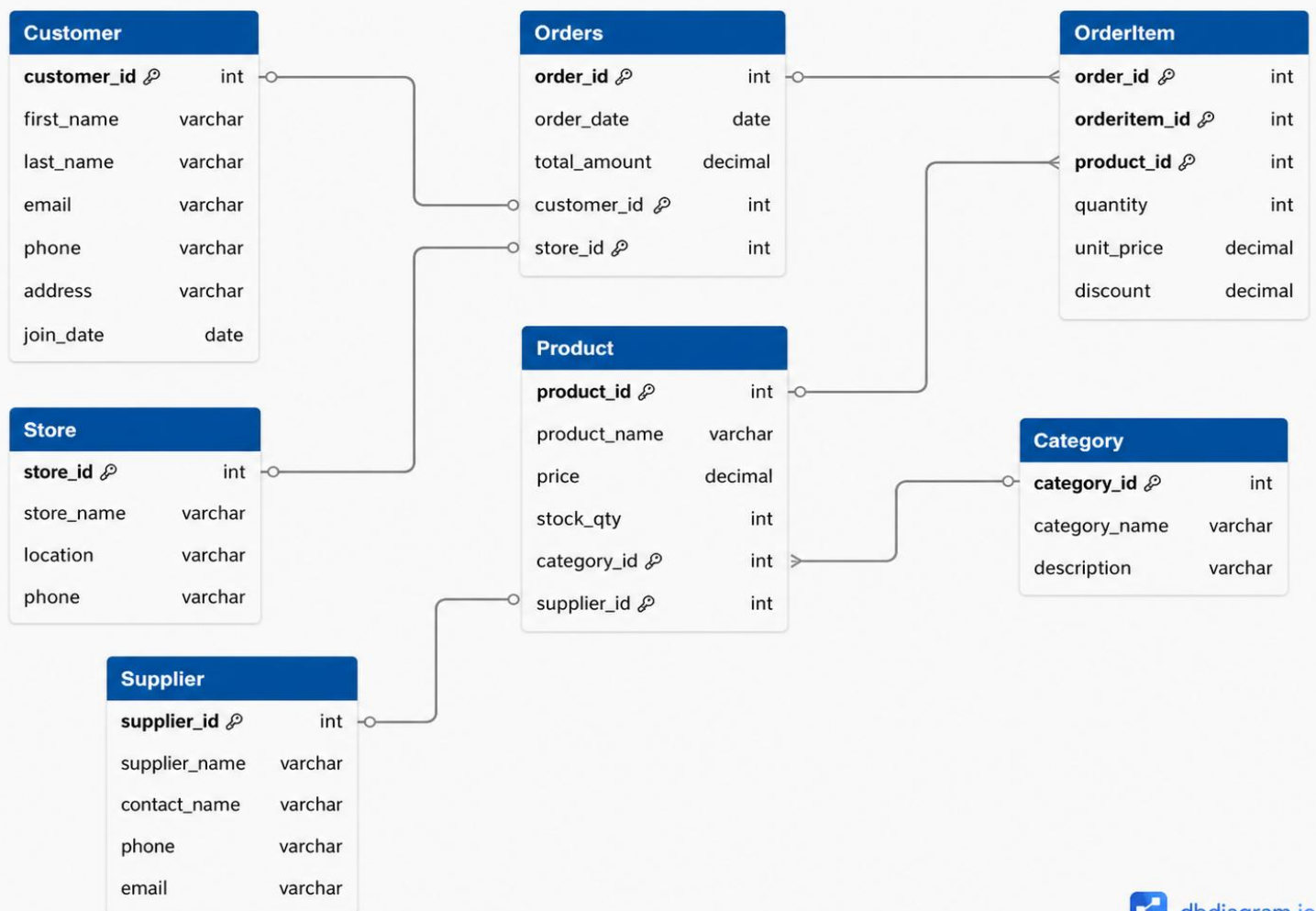
Category(category_id PK, category_name, description)

Supplier(supplier_id PK, supplier_name, contact_name, phone, email)

Product(product_id PK, product_name, price, stock_qty, category_id FK, supplier_id FK)

Orders(order_id PK, order_date, total_amount, customer_id FK, store_id FK)

OrderItem(order_id PK FK, orderitem_id PK, product_id FK, quantity, unit_price, discount)



Data Flow & System Behavior

3.1 DFD – Level 0 (Context Diagram)

1a. DFD – Level 0 (Context Diagram)

DEPI R4 - Project 5 - High-level view of the Retail Data Pipeline system

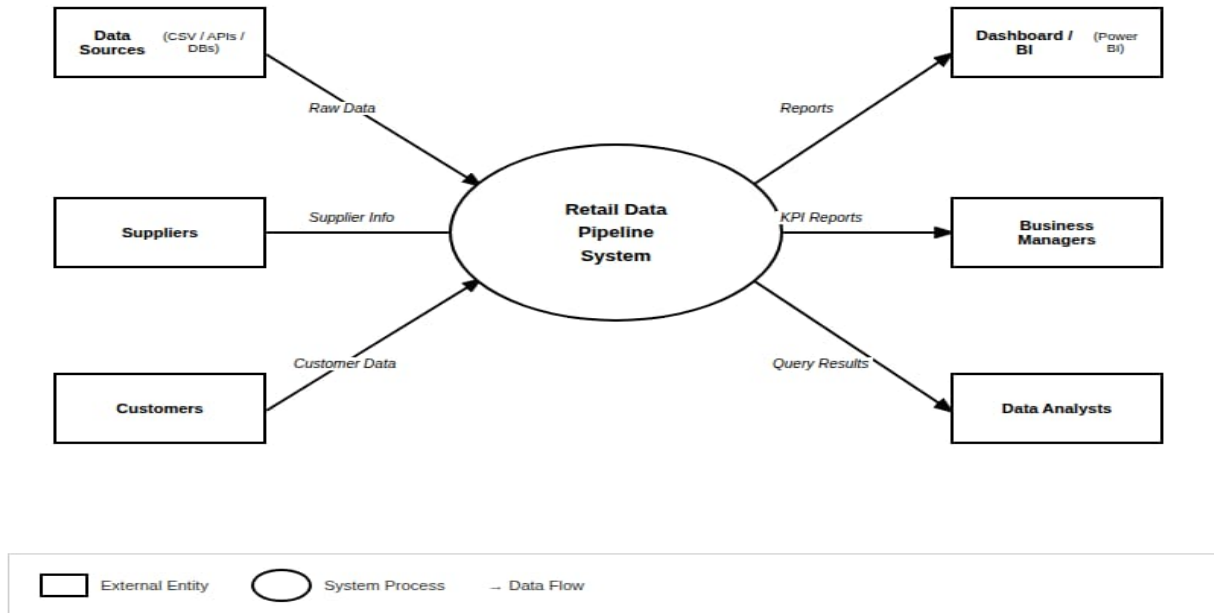


Figure 3.1 – DFD Level 0: High-level view of the Retail Data Pipeline System

The Context Diagram shows the system as a single black box. It defines who interacts with the system and what data flows in and out.

- **Inputs:** Data Sources (CSV/APIs/DBs) send raw data. Suppliers send supplier info. Customers send customer data.
- **System:** The Retail Data Pipeline System processes all inputs and produces outputs.
- **Outputs:** Dashboard/BI receives reports. Business Managers receive KPI reports. Data Analysts receive query results.

3.2 DFD – Level 1 (Retail Data Pipeline)

1b. DFD – Level 1 (Retail Data Pipeline)

DEPI R4 - Project 5 - Detailed processes, data stores, and data flows

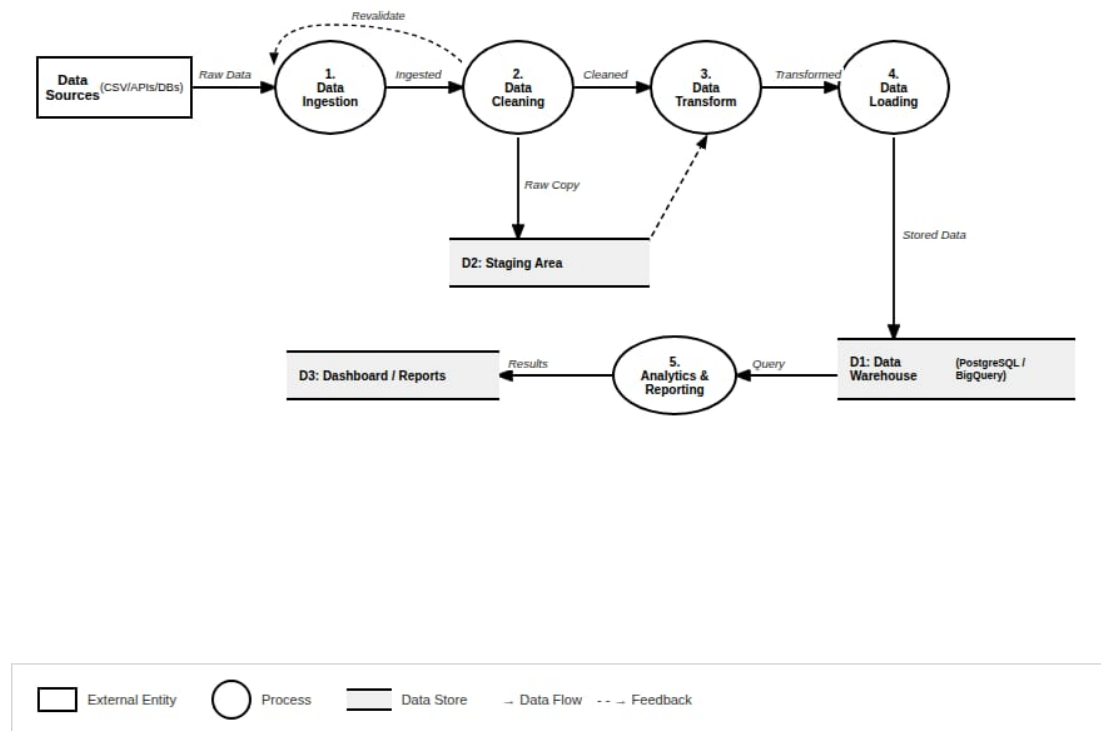


Figure 3.2 – DFD Level 1: Internal processes and data flows of the pipeline

The Level 1 DFD breaks down the system into 5 internal processes and 3 data stores, showing exactly how data moves through each pipeline stage.

- **1. Data Ingestion:** Extracts raw data from sources and stores a copy in D2: Staging Area.
- **2. Data Cleaning:** Cleans ingested data (dedup, nulls, types). A Revalidate loop handles failed records.
- **3. Data Transform:** Transforms clean data into a Star Schema (FactSales + dimension tables).
- **4. Data Loading:** Loads transformed data into D1: Data Warehouse (PostgreSQL / BigQuery).
- **5. Analytics & Reporting:** Queries D1 and sends results to D3: Dashboard / Reports (Power BI).

3.3 Sequence Diagram – View Sales Report

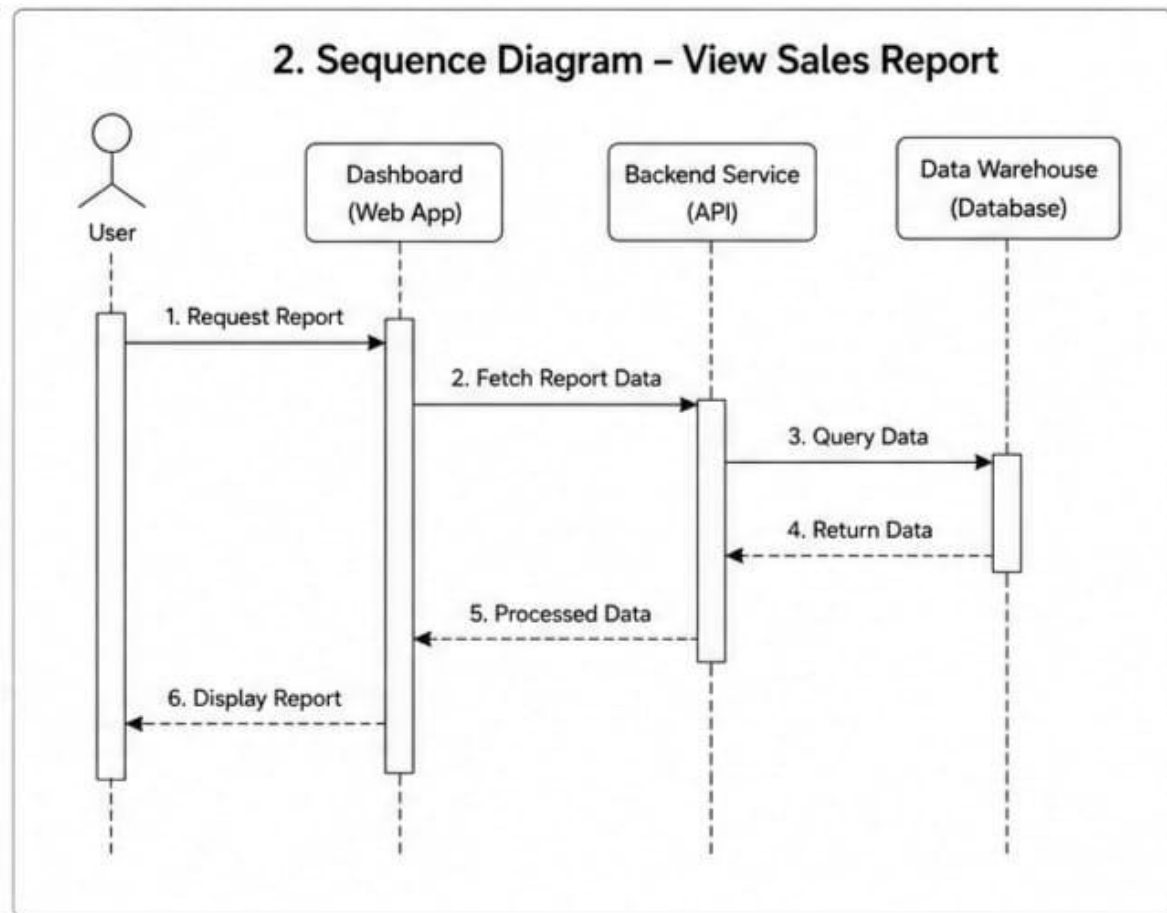


Figure 3.3 – Sequence Diagram: Interactions when a user views a sales report

Shows the time-ordered interactions between system components when a user requests a sales report.

- **1. Request Report:** User clicks "View Report" on the Dashboard.
- **2. Fetch Report Data:** Dashboard sends a request to the Backend Service (API).
- **3. Query Data:** Backend queries the Data Warehouse for the required data.
- **4. Return Data:** Data Warehouse returns the raw data to the Backend.
- **5. Processed Data:** Backend processes and sends the data back to the Dashboard.
- **6. Display Report:** Dashboard renders and displays the final report to the User.

3.4 Activity Diagram – ETL Process



3. Activity Diagram – ETL Process

DEPI R4 - Project 5 - Workflow of the retail data pipeline ETL process

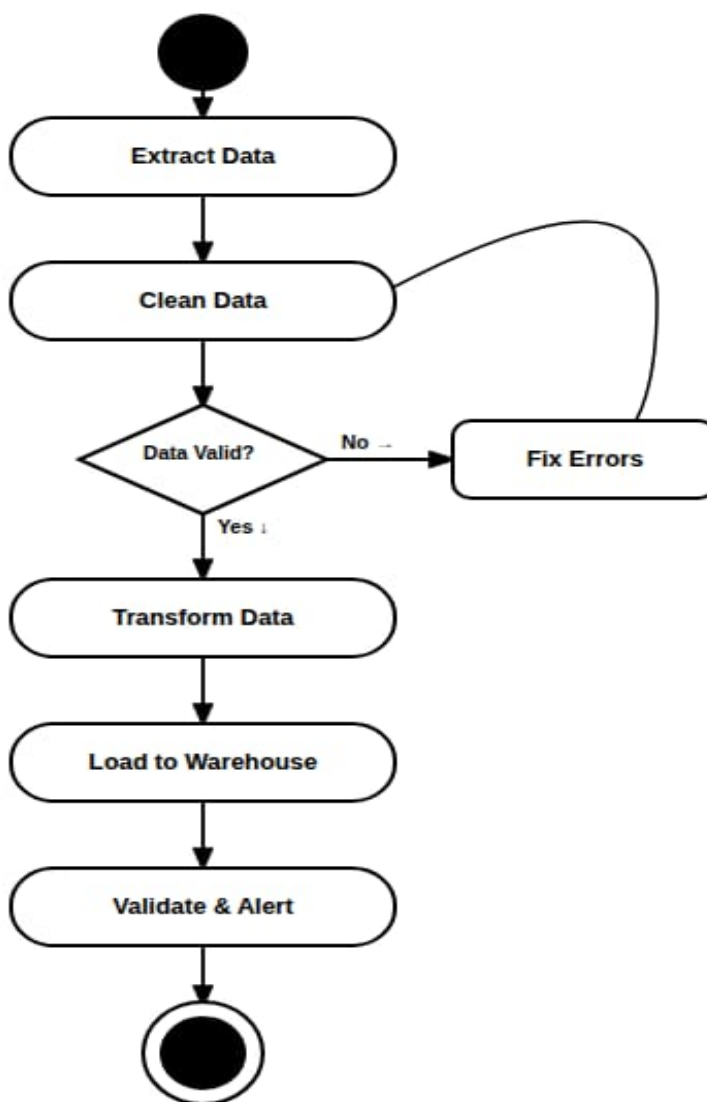


Figure 3.4 – Activity Diagram: Step-by-step ETL workflow

Shows the full ETL workflow including decision points and error handling. Each step maps to a functional requirement.

- **Extract Data:** Raw data is pulled from SQL databases and Parquet files.

- **Clean Data:** Removes duplicates, fixes nulls, and corrects data types.
- **Data Valid? (Decision):** If invalid → Fix Errors → re-clean. If valid → continue.
- **Transform Data:** Build Star Schema — FactSales + DimDate, DimProduct, DimCustomer, DimSupplier.
- **Load to Warehouse:** Load transformed data into Data Warehouse (PostgreSQL / BigQuery).
- **Validate & Alert:** Final validation. Success logged; failures trigger an Airflow alert within 5 minutes.

3.5 State Diagram – Pipeline Job States

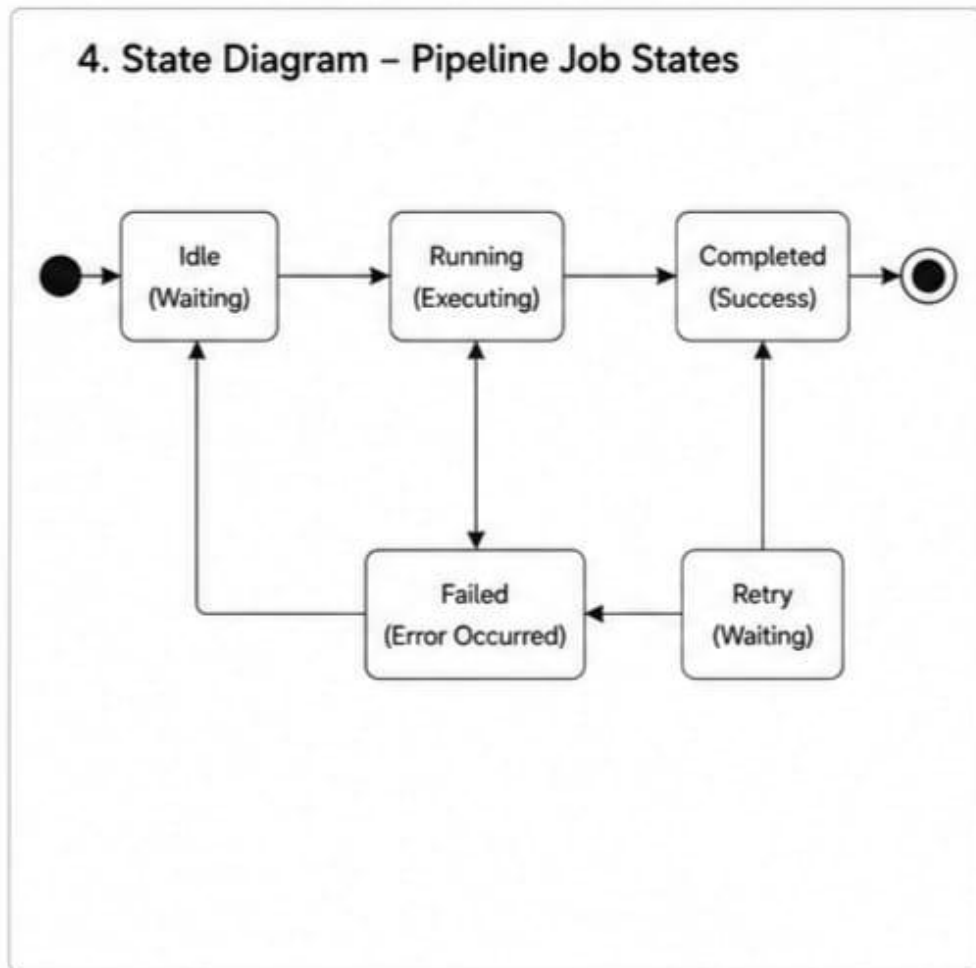


Figure 3.5 – State Diagram: Airflow DAG pipeline states

Shows all possible states of the Airflow DAG during a daily pipeline run and how it transitions between them.

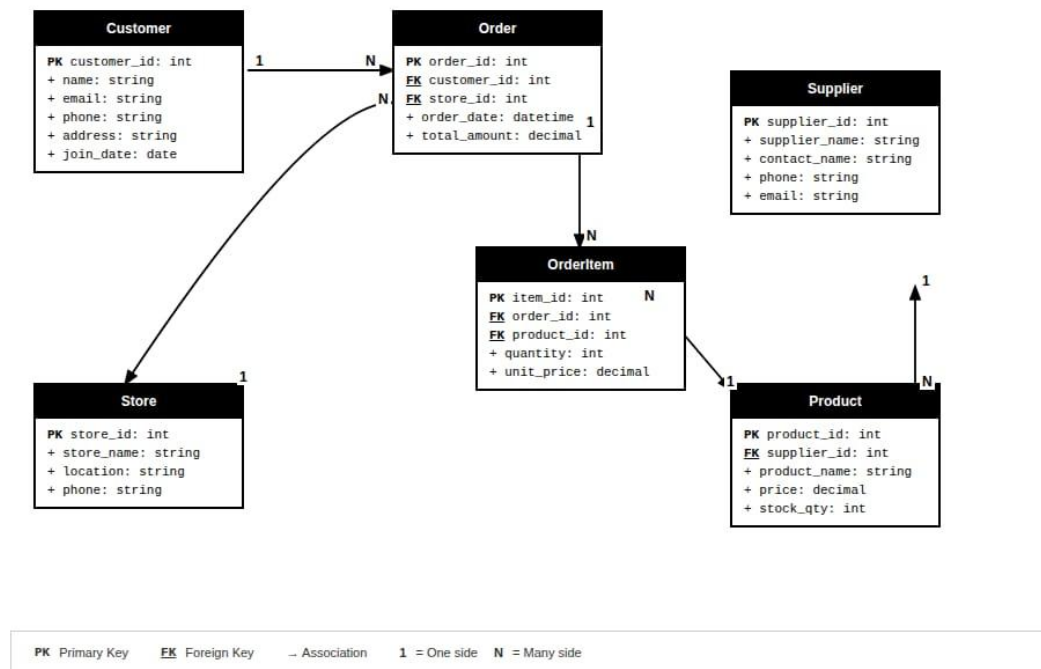
- **Idle (Waiting):** Pipeline waits for the next scheduled trigger.
- **Running (Executing):** Pipeline is actively executing its tasks.
- **Completed (Success):** All tasks passed successfully. Returns to Idle.
- **Failed (Error Occurred):** Task failed after all retry attempts. Alert sent to Admin.
- **Retry (Waiting):** Task failed but retries remain (max 3 retries per NFR-07). Returns to Running.

3.6 Class Diagram – Core Data Model

Figure 3.6 – Class Diagram: Retail data model entities and relationships

5. Class Diagram – Core Data Model

DEPI R4 - Project 5 - All 6 entities with attributes, keys, and relationships (incl. Supplier)



Defines the structure of the retail database — entities, attributes (PK/FK), and how they relate to each other.

- **Customer (1–N Order):** customer_id, name, email, phone, address, join date.
- **Order (N–1 Customer, N–1 Store):** order_id, customer_id (FK), store_id (FK), order_date, total_amount.
- **OrderItem (N–1 Order, N–1 Product):** item_id, order_id (FK), product_id (FK), quantity, unit_price.
- **Product (N–1 Supplier):** product_id, supplier_id (FK), product_name, price, stock_qty.
- **Store (1–N Order):** store_id, store_name, location, phone.
- **Supplier (1–N Product):** supplier_id, supplier_name, contact_name, phone, email

Low-Fidelity Wireframes — Design Documentation

1. Introduction

This document presents the low-fidelity wireframes designed for RetailPipe, the retail data pipeline system developed as part of Project 5 of the DEPI R4 Data Engineering Track. The wireframes define the layout, structure, and user flow of all system screens without applying any visual styling or final UI polish.

The wireframes were developed based on two primary sources:

- The Literature Review document, which defines the system goals and data engineering architecture.
- The Requirements Gathering document, which defines user stories, stakeholder roles, functional requirements, and non-functional requirements.

These wireframes serve as a communication artifact between the engineering team and stakeholders, establishing a shared understanding of the system's interface before implementation.

2. Purpose & Scope

Low-fidelity wireframes focus exclusively on:

- Layout and spatial arrangement of UI components
- Hierarchy and grouping of information
- Navigation flow between screens
- Component identification (buttons, tables, charts, forms, modals)

The scope of this document covers all 9 screens of the RetailPipe system, mapped to the 4 system actors and 9 user stories defined in the requirements document.

3. System Actors & Screen Access

The system supports four distinct actors, each with a defined role and access level. The following table summarizes which screens each actor can access:

Actor	Role	Accessible Screens	User Stories
Data Engineer	Pipeline builder & maintainer	0, 1, 2, 3, 6	US-01, US-02, US-03, US-08
Data Analyst	Consumer of processed data	0, 1, 4, 8	US-04, US-05
Business Manager	Strategic decision-maker	0, 1, 5, 8	US-06, US-07
System Administrator	Infrastructure & access control	0, 1, 2, 3, 6, 7 + all	US-08, US-09

4. User Flow Overview

All users begin at Screen 0 (Login). After successful authentication, the system routes the user to Screen 1 (Dashboard) as the default landing page. Navigation between screens is handled through the top navigation bar, which adapts based on the user's role.

The primary navigation flow is as follows:

- 1. Screen 0 (Login) → Screen 1 (Dashboard)
- 2. Screen 1 (Dashboard) → Screen 2, 3, 4, 5, 8 via top navbar
- 3. Screen 3 (Pipelines) → Screen 6 (Task Detail) via clicking a DAG node
- 4. Screen 1 (Dashboard) → Screen 7 (User Management) — Admin only, via Settings

5. Wireframe Conventions

The following notation is used consistently across all wireframe screens in this document:

Element	Meaning
[Label]	Interactive component: button, dropdown, input field, or tab
Dashed box	Content placeholder — chart, image, or dynamic data area
Solid box	Static UI component — card, panel, or container
—— line ——	Separator or visual divider between sections
→ Arrow	Navigation trigger or data flow direction
Navbar (top)	Global navigation bar — present on all screens after login
Sidebar (left)	Section-level navigation — icons representing main areas
Badge	Status indicator (Active / Error / Idle / Warning)

6. Screen-by-Screen Wireframe Descriptions

Each screen below is described in terms of its purpose, components, layout structure, and relationship to the defined user stories.

Login page (entry point) *All actors · Authentication gate before accessing the system*

RetailPipe — DEPI R4

Sign in
Enter your credentials to access the dashboard

Email address
user@retailpipe.com

Password
.....

[Forgot password?](#)

Sign in →

Role determines access level:

[Data Engineer](#) [Data Analyst](#) [Business Mgr](#)
[Sys Admin](#)

Screen 0 — Login

Screen info	
Screen number	Screen 0
Screen name	Login
Actor(s)	All actors (Data Engineer, Data Analyst, Business Manager, System Administrator)
User stories	N/A — authentication gate

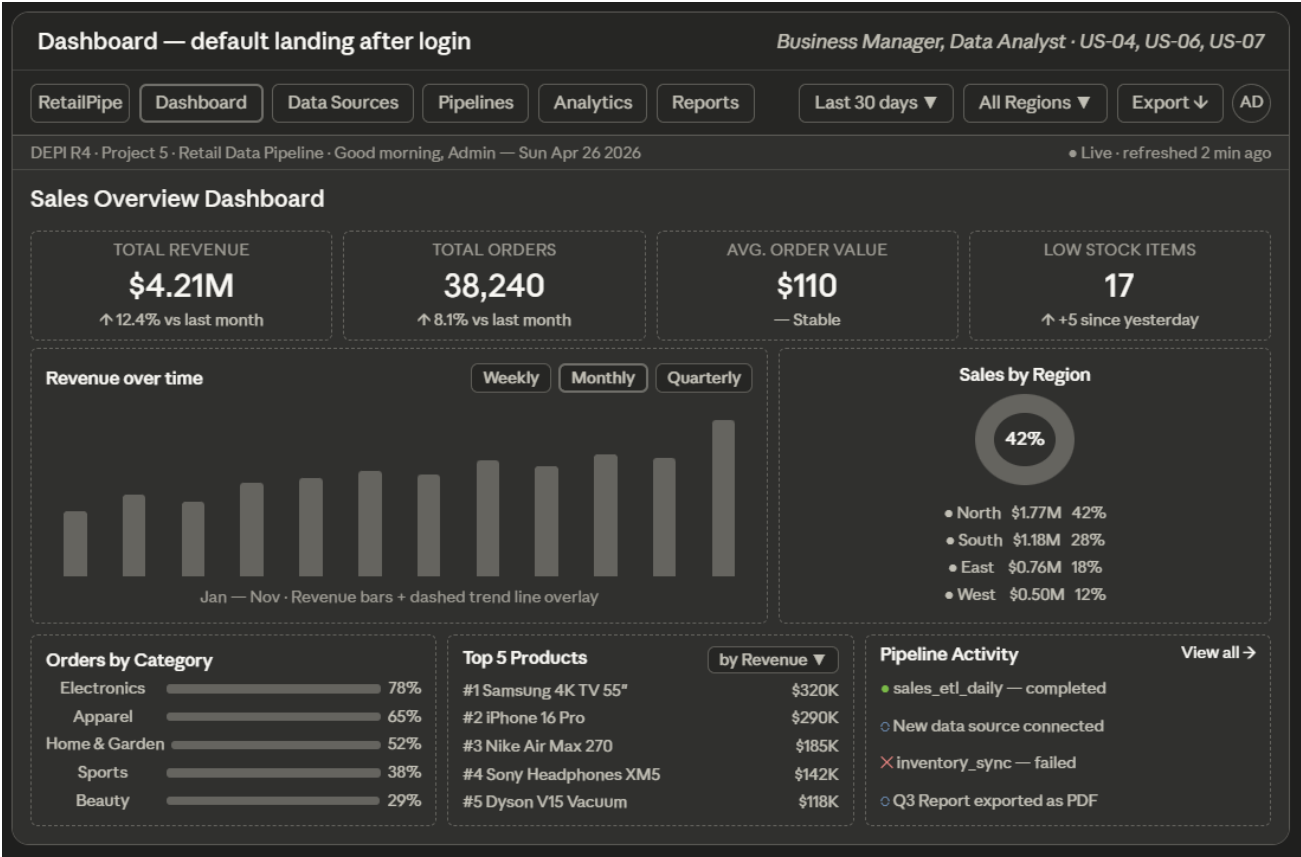
Purpose

The login screen is the system's entry point. It authenticates users and routes them to the Dashboard based on their assigned role. No content is accessible before successful authentication, enforcing the access control requirements defined in US-09.

Layout structure

- [Brand logo / system name] — Identifies the system — 'RetailPipe — DEPI R4'
- [Email address field] — Input for user's registered email
- [Password field] — Masked input for password
- [Forgot password link] — Secondary link — password recovery flow

- [Sign in button] — Primary action — submits credentials and triggers authentication
- [Role reference panel] — Informs users of the 4 role types: Data Engineer, Data Analyst, Business Manager, Sys Admin



Screen 1 — Dashboard — Sales Overview

Screen info	
Screen number	Screen 1
Screen name	Dashboard — Sales Overview
Actor(s)	Business Manager, Data Analyst
User stories	US-04, US-06, US-07

Purpose

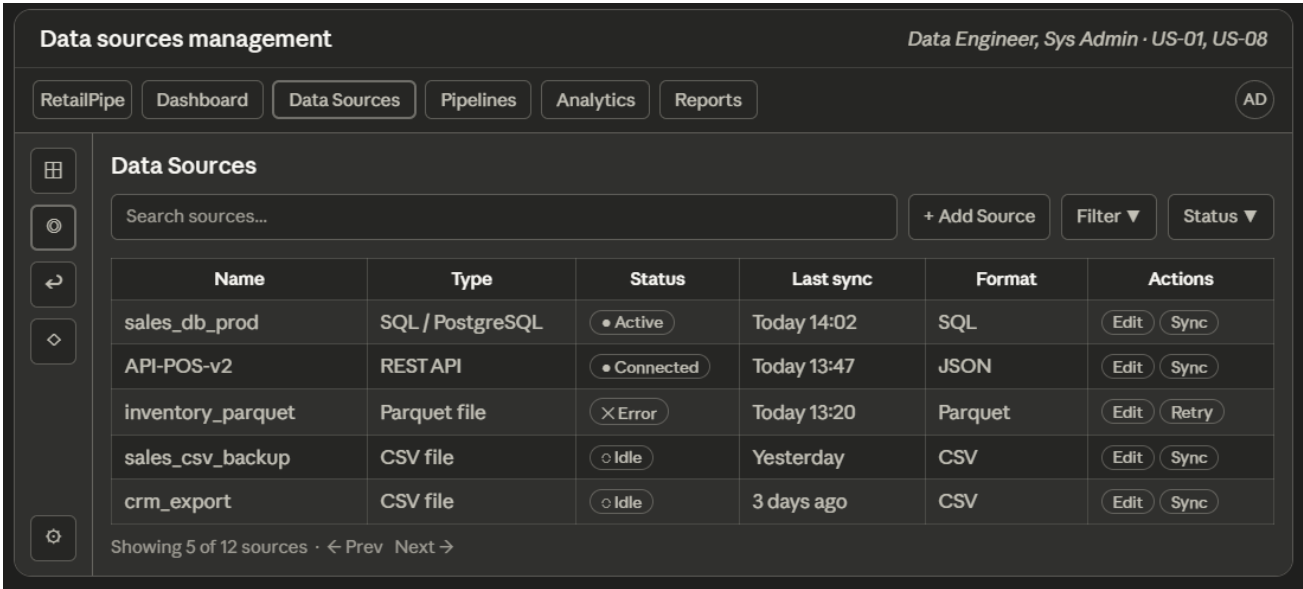
The Dashboard is the default landing page after login. It provides a high-level summary of the most critical business KPIs, revenue trends, regional sales distribution, product performance, and pipeline activity. It is the central hub from which users navigate to all other screens.

Layout structure

- [Top navigation bar] — Logo + nav links (Dashboard, Data Sources, Pipelines, Analytics, Reports, Inventory) + date filter + region filter + export button + user avatar
- [Subbar] — Breadcrumb context (DEPI R4 · Project 5) + live data status indicator
- [Page title] — 'Sales Overview Dashboard'

- [KPI row (4 cards)] — Total Revenue | Total Orders | Avg. Order Value | Low Stock Items — each with current value and month-over-month change
- [Revenue over time (bar chart)] — Monthly revenue bars from Jan to Nov + dashed trend line overlay — toggle: Weekly / Monthly / Quarterly
- [Sales by region (donut chart)] — Proportional split across North, South, East, West — legend with revenue and percentage
- [Orders by category (horizontal bars)] — Category breakdown: Electronics, Apparel, Home & Garden, Sports, Beauty
- [Top 5 products (list)] — Ranked by revenue — product name + value
- [Pipeline activity feed] — Last 4 pipeline events with status icons: completed / connected / failed / exported

Screen 2 — Data Sources



Screen info	
Screen number	Screen 2
Screen name	Data Sources
Actor(s)	Data Engineer, System Administrator
User stories	US-01, US-08

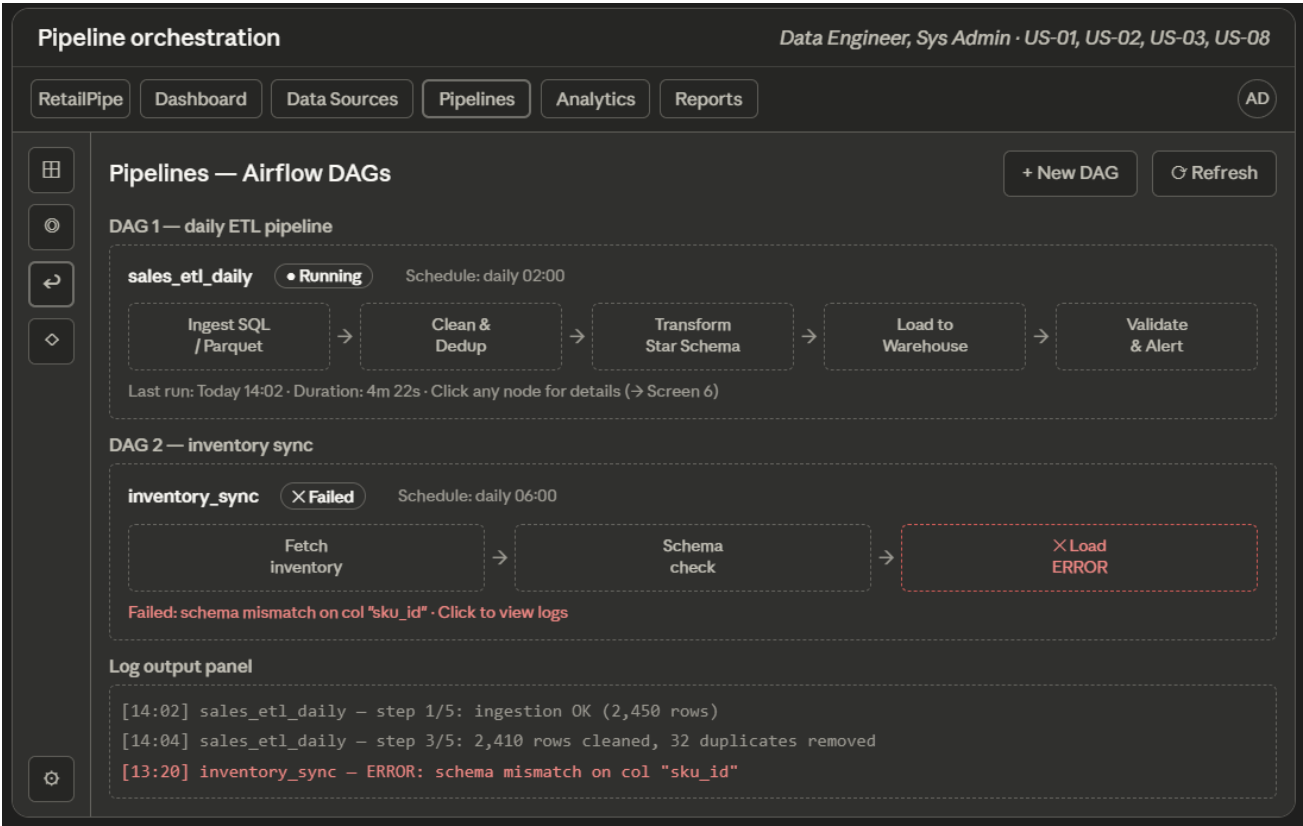
Purpose

The Data Sources screen allows the Data Engineer to manage all connections to external data sources including SQL databases, Parquet files, REST APIs, and CSV files. It provides visibility into connection status, last sync time, and file format, and supports adding new sources or retrying failed ones.

Layout structure

- [Top navigation bar] — Same global navbar — 'Data Sources' tab active

- [Left sidebar] — Icon-based section navigation — Data Sources icon active
- [Page title] — 'Data Sources'
- [Search bar] — Filter sources by name or keyword
- [Add Source button] — Opens a form/modal to register a new data source connection
- [Filter dropdowns] — Filter by status (Active / Error / Idle) or connection type
- [Data sources table] — Columns: Name | Type | Status | Last Sync | Format | Actions (Edit, Sync, Retry)
- [Pagination bar] — Showing N of total · Previous / Next navigation



Screen 3 — Pipelines — Airflow DAGs

Screen info	
Screen number	Screen 3
Screen name	Pipelines — Airflow DAGs
Actor(s)	Data Engineer, System Administrator
User stories	US-01, US-02, US-03, US-08

Purpose

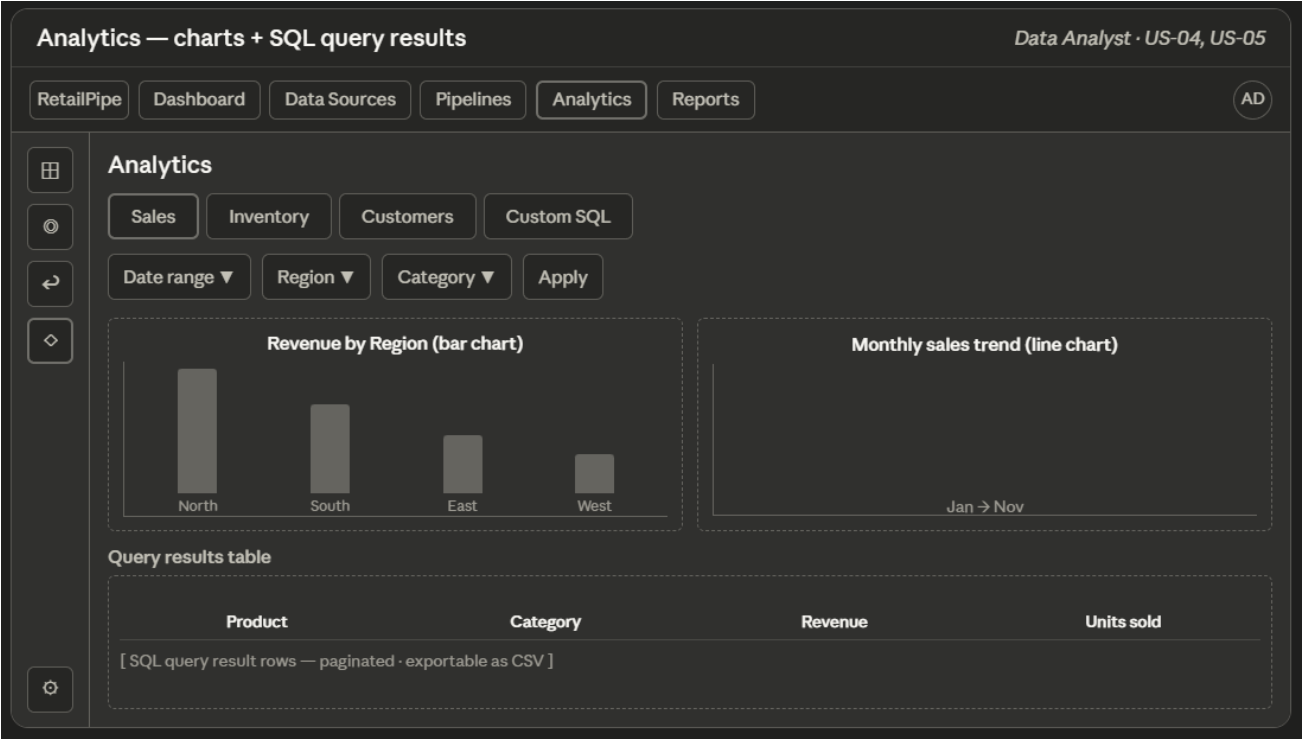
The Pipelines screen provides a visual representation of all Apache Airflow DAGs running in the system. It displays the stages of each pipeline as a node graph, shows run status and scheduling information, and surfaces a real-time log output panel. Clicking a node opens the Pipeline Task Detail modal (Screen 6).

Layout structure

- [Top navigation bar] — Global navbar — 'Pipelines' tab active
- [Left sidebar] — Pipelines icon active

- [Page title + action buttons] — 'Pipelines — Airflow DAGs' + [+ New DAG] + [Refresh]
- [DAG panel — sales_etl_daily] — Status: Running | Schedule: daily 02:00 | Nodes: Ingest SQL/Parquet → Clean & Dedup → Transform Star Schema → Load Warehouse → Validate & Alert
- [DAG panel — inventory_sync] — Status: Failed | Schedule: daily 06:00 | Nodes: Fetch Inventory → Schema Check → Load (ERROR) — error message displayed below
- [Log output panel] — Scrollable real-time log with colour-coded entries: success (grey), error (red), info

Screen 4 — Analytics



Screen info	
Screen number	Screen 4
Screen name	Analytics
Actor(s)	Data Analyst
User stories	US-04, US-05

Purpose

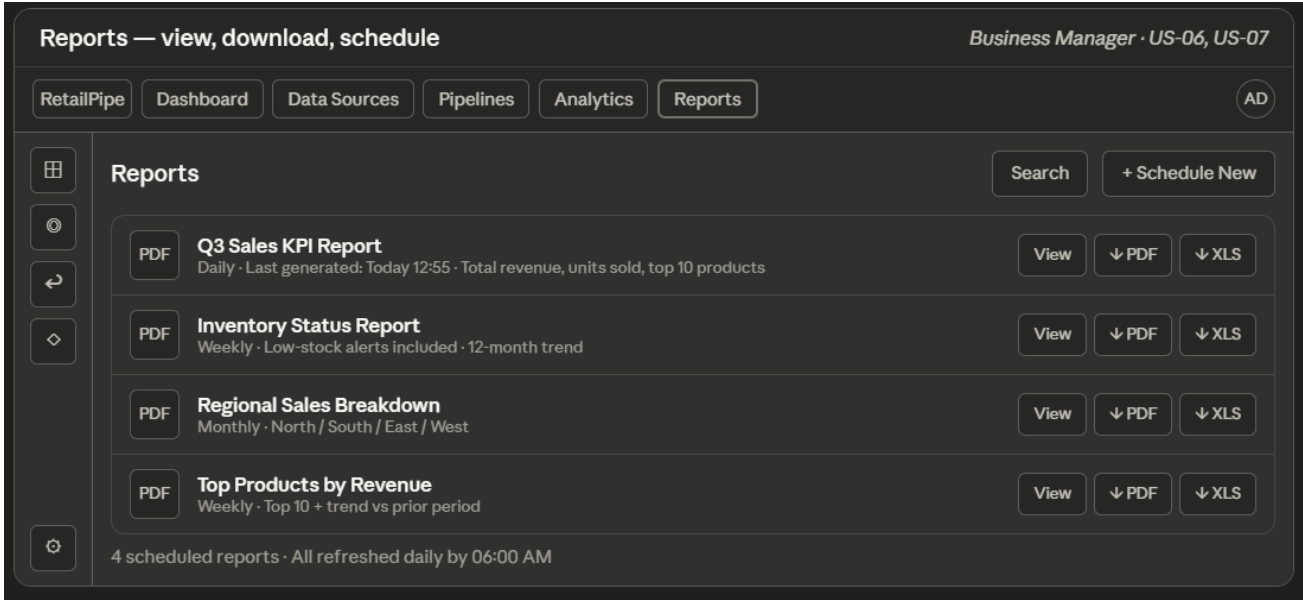
The Analytics screen provides the Data Analyst with interactive charts and SQL query result tables. It supports tab-based navigation between Sales, Inventory, Customer, and Custom SQL views, with filters for date range, region, and category. Results are displayed in paginated tables and can be exported as CSV.

Layout structure

- [Top navigation bar] — Global navbar — 'Analytics' tab active
- [Left sidebar] — Analytics icon active
- [Page title] — 'Analytics'

- [**Tab bar**] — Sales | Inventory | Customers | Custom SQL
- [**Filter bar**] — Date range dropdown | Region dropdown | Category dropdown | Apply button
- [**Revenue by Region (bar chart)**] — Vertical bars for North, South, East, West with axis labels
- [**Monthly Sales Trend (line chart)**] — Jan to Nov data points — trend line overlay
- [**Query results table**] — Columns: Product | Category | Region | Revenue | Units sold | Avg price — paginated, exportable as CSV

Screen 5 — Reports



Screen info	
Screen number	Screen 5
Screen name	Reports
Actor(s)	Business Manager
User stories	US-06, US-07

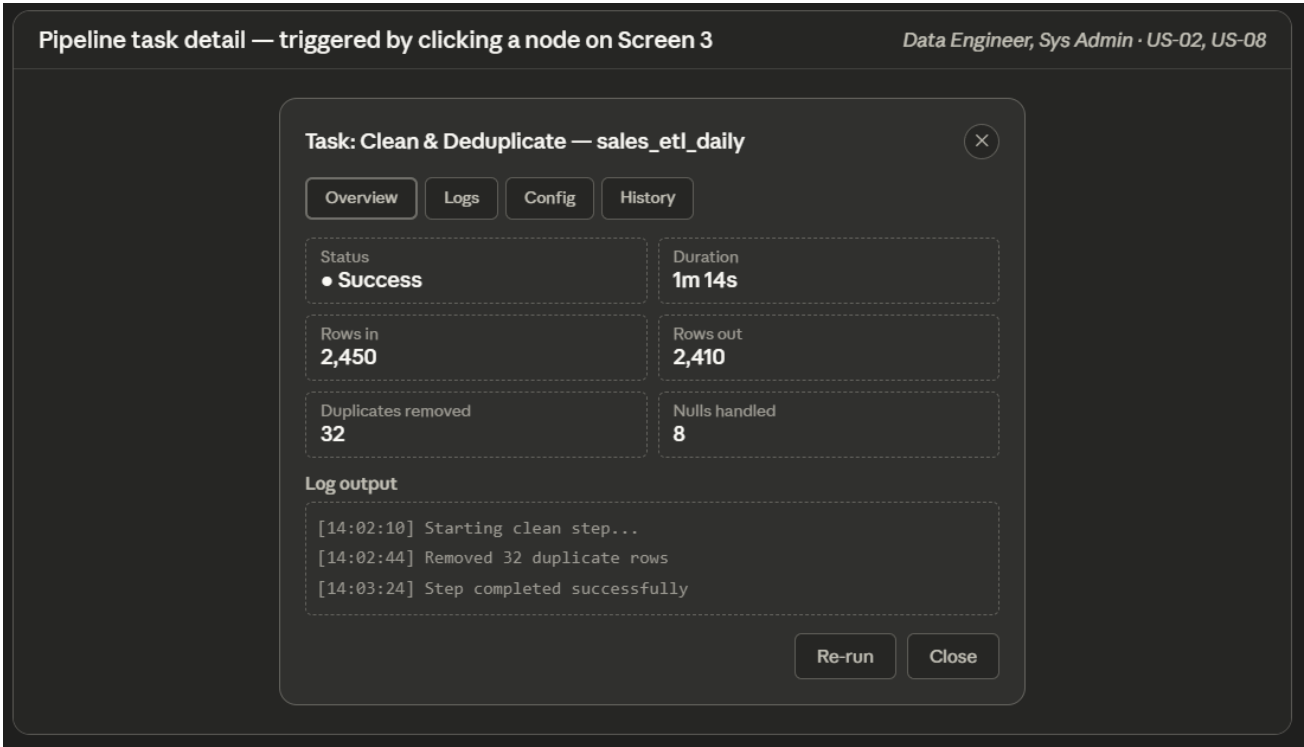
Purpose

The Reports screen provides the Business Manager with access to all scheduled and on-demand reports. Each report can be viewed, downloaded as PDF or Excel, or scheduled for periodic delivery. Reports cover KPIs, inventory, regional sales, and top products.

Layout structure

- [**Top navigation bar**] — Global navbar — 'Reports' tab active
- [**Left sidebar**] — Reports icon active
- [**Page title + actions**] — 'Reports' + Search field + [+ Schedule New] button
- [**Report list**] — Each row includes: icon | report name | metadata (frequency, last generated, content description) | [View] [Download PDF] [Download XLS] buttons
- [**Reports included**] — Q3 Sales KPI Report | Inventory Status | Regional Sales Breakdown | Top Products by Revenue | Customer Segmentation
- [**Footer note**] — Refresh schedule info — all reports refreshed daily by 06:00 AM

Screen 6 — Pipeline Task Detail (Modal)



Screen info	
Screen number	Screen 6
Screen name	Pipeline Task Detail (Modal)
Actor(s)	Data Engineer, System Administrator
User stories	US-02, US-08

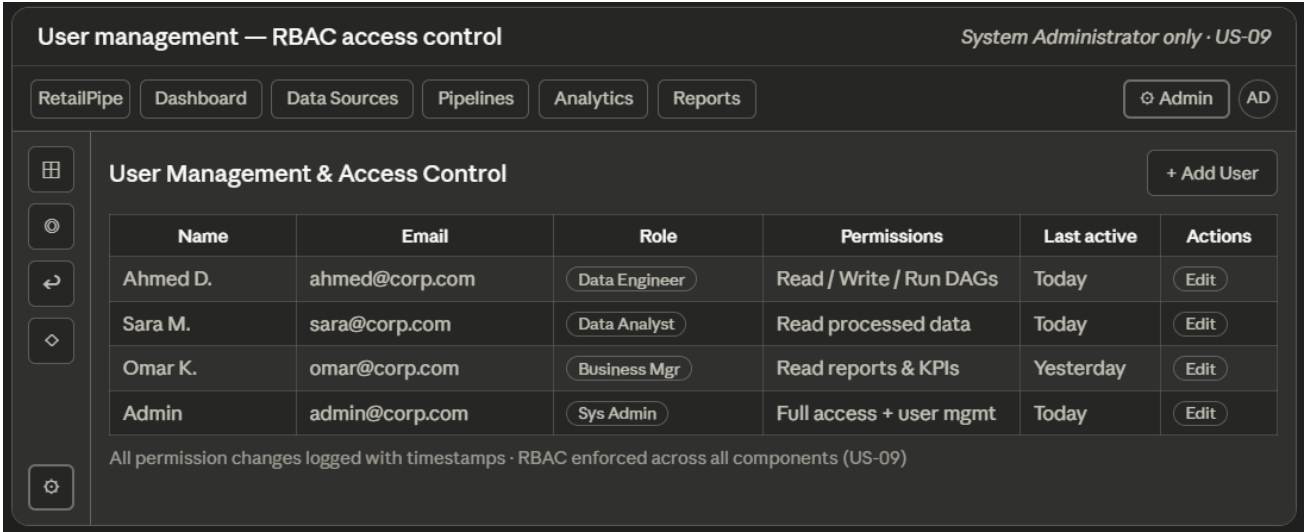
Purpose

The Pipeline Task Detail modal appears as an overlay when a user clicks any individual node within a DAG on Screen 3. It provides detailed execution statistics, a log viewer, and a re-run action for the selected pipeline task.

Layout structure

- [Overlay / dimmed background] — The Pipelines screen (Screen 3) is dimmed behind the modal
- [Modal header] — Task name + DAG name + [X] close button
- [Tab bar] — Overview | Logs | Config | History
- [Stats grid (2 x 3)] — Status | Duration | Rows in | Rows out | Duplicates removed | Nulls handled
- [Log output area] — Scrollable timestamped log entries for the selected task
- [Action buttons] — [Re-run] and [Close] — bottom right aligned

Screen 7 — User Management — RBAC



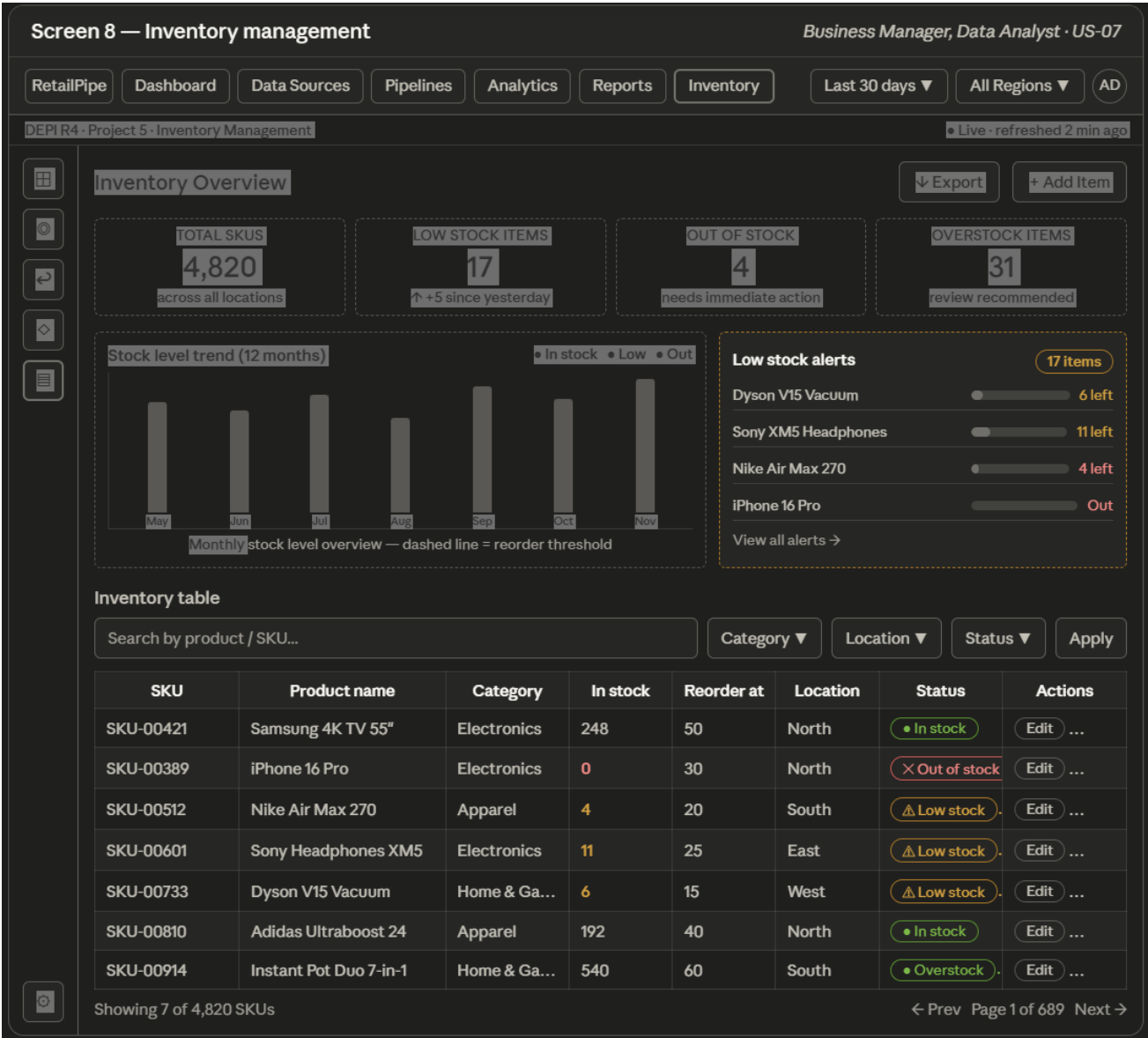
Screen info	
Screen number	Screen 7
Screen name	User Management — RBAC
Actor(s)	System Administrator only
User stories	US-09

Purpose

The User Management screen is accessible only to the System Administrator. It provides full control over user accounts and role-based access control (RBAC), allowing the Admin to add users, assign roles, modify permissions, and revoke access. All changes are logged with timestamps.

Layout structure

- [Top navigation bar] — Global navbar — Admin settings section active
- [Left sidebar] — Settings (gear) icon active
- [Page title + action] — 'User Management & Access Control' + [+ Add User] button
- [Users table] — Columns: Name | Email | Role | Permissions | Last Active | Actions (Edit, Revoke)
- [Role permissions summary] — 4 role cards showing each role's permission scope at a glance



Screen 8 — Inventory Management

Screen info	
Screen number	Screen 8
Screen name	Inventory Management
Actor(s)	Business Manager, Data Analyst
User stories	US-07

Purpose

The Inventory Management screen gives the Business Manager and Data Analyst a complete view of stock levels across all products and locations. It surfaces low-stock and out-of-stock alerts, a 12-month stock trend chart, and a detailed filterable inventory table with reorder actions.

Layout structure

[**Top navigation bar**] — Global navbar — 'Inventory' tab active

[**Left sidebar**] — Inventory icon active

[**Page title + actions**] — 'Inventory Overview' + [Export] + [+ Add Item]

[**KPI row (4 cards)**] — Total SKUs | Low Stock Items (warning colour) | Out of Stock (alert colour) | Overstock Items

[**Stock level trend chart (bar)**] — 7-month bar chart with dashed reorder threshold line overlay

[**Low stock alerts panel**] — Top 4 low-stock or out-of-stock items with visual progress bars showing remaining stock vs. reorder threshold

[**Filter bar**] — Search by SKU/product + Category / Location / Status dropdowns + Apply

[**Inventory table**] — Columns: SKU | Product Name | Category | In Stock | Reorder At | Location | Status | Actions (Edit, Reorder)

[**Pagination bar**] — Showing N of 4,820 SKUs — Previous / Next

7. Non-Functional Requirements Coverage

The wireframe design addresses the non-functional requirements defined in the requirements document as follows:

NFR Category	Requirement	Wireframe Implementation
Performance	Query < 10s (US-04)	Analytics screen: noted in footer. Data refreshed daily by 06:00 AM.
Security	RBAC across all components (US-09)	Login gate (Screen 0) + User Management (Screen 7) + role-restricted navigation.
Reliability	Failure alerts within 5 min (US-08)	Pipeline Activity feed on Dashboard + Log panel on Screen 3 + Email alert noted in logs.
Scalability	Handles growing data volumes	Paginated tables across all list screens (Data Sources, Inventory, Analytics).
Usability	Accessible to all 4 actor types	Role-based navigation: each actor sees only relevant screens.
Maintainability	Logs retained 30 days (US-08)	Log viewer on Screen 3 and Screen 6 with timestamped entries.
Availability	Real-time pipeline status	Dashboard activity feed + '● Live · refreshed 2 min ago' subbar indicator.

8. Summary

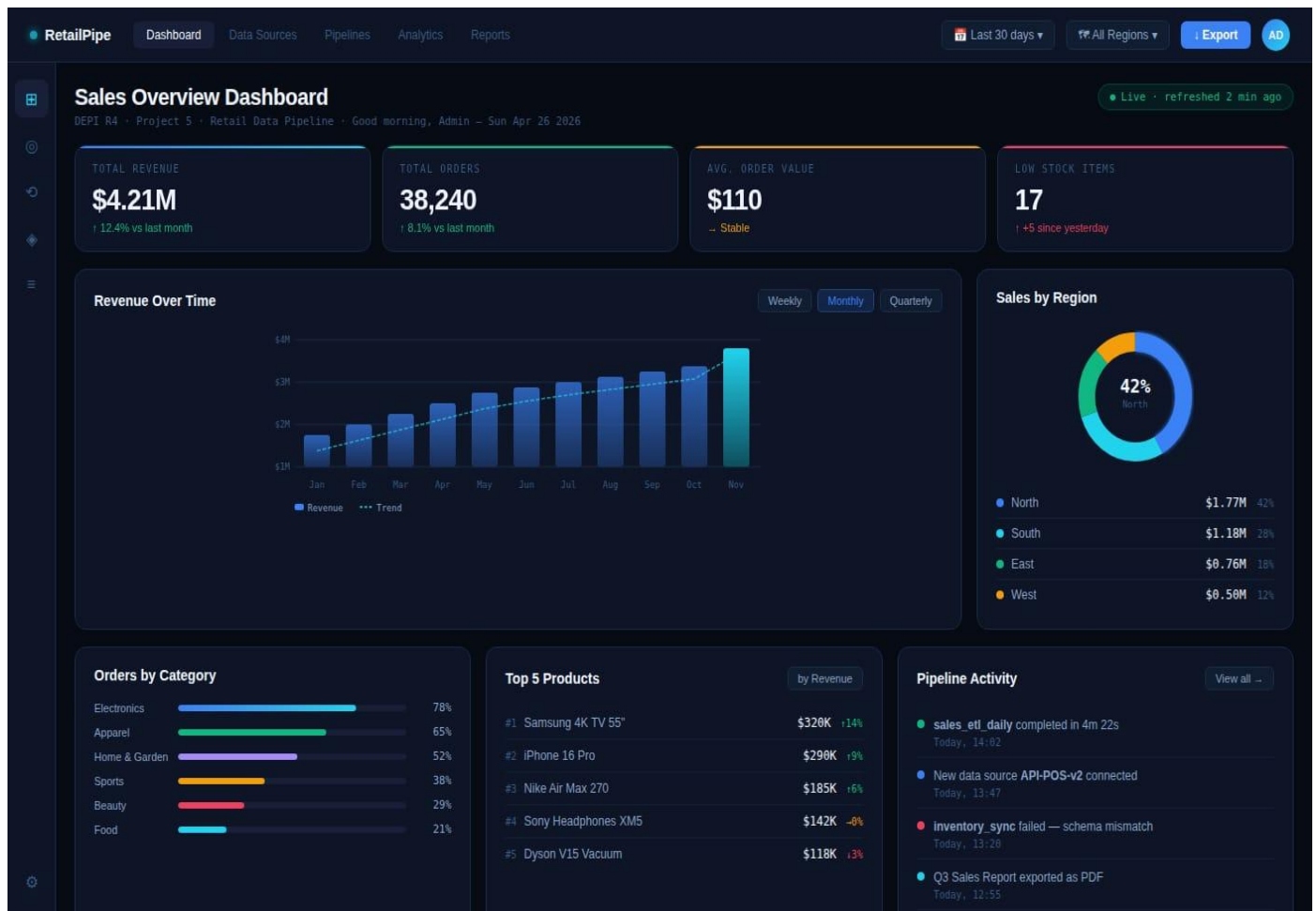
The RetailPipe wireframes cover 9 screens across the full user journey, from authentication through pipeline management, data analysis, reporting, and inventory monitoring. Each screen is grounded in the user stories and functional requirements defined in the project's requirement gathering document.

The design ensures:

- All 4 system actors have clearly defined access paths and screen coverage.
- All 9 user stories (US-01 through US-09) are addressed by at least one screen.

- Non-functional requirements (performance, security, reliability, scalability, usability) are reflected in structural decisions visible at the wireframe level.
- The navigation flow is simple and consistent — a fixed navbar and sidebar provide predictable wayfinding across all screens.

These wireframes are intended for inclusion in the Project 5 graduation report as the UX design section, and serve as the foundation for the high-fidelity design and front-end implementation phases.



9. High-Fidelity Mockup — Dashboard

A high-fidelity dashboard mockup was designed to represent the final user interface of the system.

The dashboard displays key business metrics and insights in a clear and structured way.

Main components:

- KPI cards: Total Revenue, Total Orders, Average Order Value, Low Stock Items
- Revenue over time chart
- Sales by region chart
- Orders by category
- Top products
- Pipeline activity status

The dashboard helps users monitor performance and make data-driven decisions quickly.

10. UI/UX Design Guidelines

Design Principles:

- Simple and clear layout
- Important data shown first (KPIs)
- Consistent design across screens

Color Scheme:

- Dark theme
- Blue: data
- Green: positive
- Red: errors
- Orange: warnings

Typography:

- Large text for KPIs
- Medium for titles
- Small for details

Accessibility:

- High contrast
- Clear readable text

System Deployment & Integration

5.1 Technology Stack

The system is built using a modern data engineering technology stack to support data ingestion, processing, storage, and orchestration.

Programming Language

- **Python:** Used for implementing ETL processes including data extraction, cleaning, and transformation.

Data Processing

- **Pandas:** Used for handling structured data, cleaning datasets, and performing transformations.

Data Sources

- **SQL Databases:** Provide transactional sales data as raw input to the pipeline.
- **Parquet Files:** Provide structured columnar data optimized for analytics.

Workflow Orchestration

- **Apache Airflow:** Used to automate, schedule, and monitor ETL pipeline tasks via DAGs.

Data Storage (Data Warehouse)

- **PostgreSQL:** Used as the centralized data warehouse to store processed and structured data.

Containerization

- **Docker:** Used to containerize the application and ensure consistency across different environments.

Data Query & Access

- **SQL:** Used to query and retrieve data from the data warehouse for analysis and validation.

5.2 Deployment Diagram

The Deployment Diagram below describes how software components are distributed across hardware nodes and environments. The system consists of four main nodes: Data Source, Cloud/Server Environment (Docker), Data Warehouse, and Data Access layer.

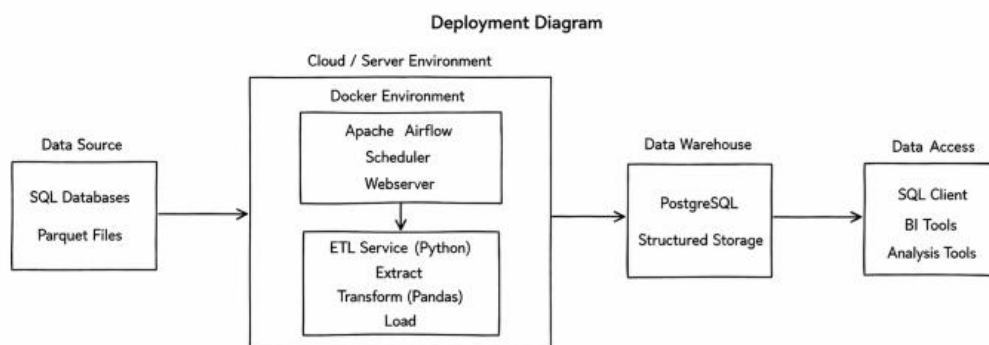


Figure 5.2 – Deployment Diagram

Node Description

- **Data Source Node:** Contains SQL Databases and Parquet Files as raw data inputs.
- **Cloud / Server Environment:** Docker containers hosting Apache Airflow (Scheduler + Webserver) and the ETL Service (Python – Extract, Transform via Pandas, Load).
- **Data Warehouse Node:** PostgreSQL database for structured storage of processed data.
- **Data Access Node:** SQL Client, BI Tools (Power BI), and Analysis Tools for querying and reporting.

5.3 Component Diagram

The Component Diagram shows the high-level system components and their dependencies within the Retail Data Pipeline System.

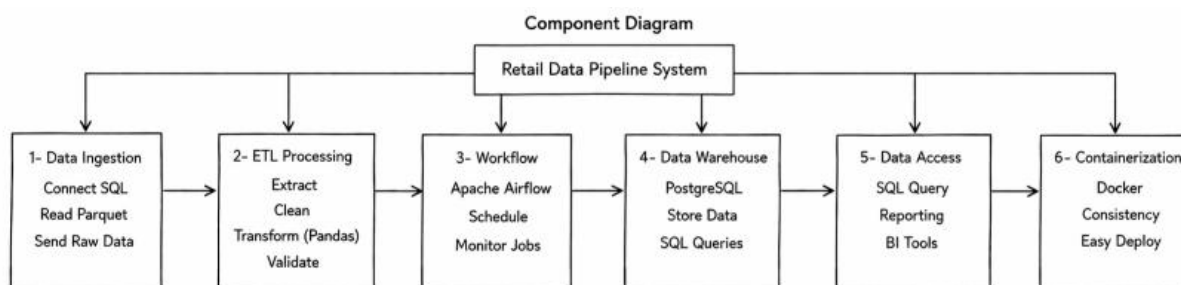


Figure 5.3 – Component Diagram

Components

- **1. Data Ingestion:** Connects to SQL databases, reads Parquet files, and sends raw data downstream.
- **2. ETL Processing:** Extracts, cleans, transforms using Pandas, and validates the data.
- **3. Workflow Orchestration:** Apache Airflow schedules, monitors, and manages pipeline jobs.

- **4. Data Warehouse:** PostgreSQL stores processed data and supports SQL queries.
- **5. Data Access:** Provides SQL query interface, reporting, and BI Tools integration.
- **6. Containerization:** Docker ensures consistency and easy deployment across environments.
-

5.4 Integration Between Components

The following describes how the pipeline components integrate and interact with each other end-to-end:

- **Apache Airflow** schedules pipeline jobs daily via configured DAGs.
- **Data Ingestion** extracts data from SQL databases and Parquet files as raw inputs.
- **Python ETL scripts** clean, deduplicate, and transform the raw data using Pandas.
- **Processed data** is loaded into PostgreSQL / BigQuery for storage.
- **Power BI** connects to the data warehouse to generate dashboards and KPI reports.

5.5 Deployment Strategy

The system is deployed using Docker Compose, where each service runs in a separate container for isolation, consistency, and ease of management.

Docker Compose Services

- **Airflow Webserver:** Provides the UI for monitoring and managing DAGs.
- **Airflow Scheduler:** Triggers and schedules pipeline runs based on defined intervals.
- **PostgreSQL Container:** Hosts the data warehouse and Airflow metadata database.
- **ETL Service Container:** Runs the Python ETL scripts for extraction, transformation, and loading.

Benefits of Docker Compose Deployment

- Consistency across development, testing, and production environments.
- Easy service isolation – each component runs independently.
- Simple scaling by adding more containers as needed.
- Straightforward setup with a single docker-compose.yml configuration file.

6-Additional Deliverables

1.1 Introduction:

This section presents the additional deliverables for the RetailPipe system, including API documentation, testing and validation strategy, and deployment approach. These components complement the main system design by providing deeper insights into how the pipeline would operate in a real-world environment, ensuring data reliability, system scalability, and proper integration between components.

1. API Document:

The API layer serves as an abstraction over the Data Warehouse (PostgreSQL/Big Query), provides structured access to processed retail data such as sales, products, and customer insights generated by the ETL pipeline. A logical backend service enables seamless communication between user interfaces such as (dashboards, analytics, and reports and the Data Warehouse).

2. API Endpoint:

An API endpoint is a specific URL where an API receives requests and sends responses. Each endpoint combines a resource path with an **HTTP method** (GET, POST, PUT, DELETE) to describe an action. Together, they form a complete instruction for the server that details what you want to do and where.

HTTP Method	Purpose	Example Endpoint
GET	Retrieve Data	GET /Api/products
POST	Create Data	POST /Api/products
PUT	Update Data (Full)	PUT /Api/products
PATCH	Update Data (Partial)	PATCH /Api/products

API Endpoints Specification

1	GET	/api/dashboard/kpis	Dashboard KPI Endpoint
2	GET	/api/analytics/sales	Sales Analytics Endpoint
3	GET GET	/api/inventory /api/inventory/low-stock	Inventory Monitoring Endpoints
4	GET	/api/pipelines/status	Data Pipeline (Airflow) Status Endpoint
5	GET GET	/api/reports /api/reports/download/{report_id}	Reports Management Endpoints
6	GET POST	/api/users /api/users/role	User Management (RBAC) Endpoints

- This figure shows sample API endpoints and their corresponding HTTP methods used in the system.

API Requests and Responses

- Request: The client sends data or parameters to the API.
- Processing: The API validates, executes logic, and interacts with the server.

Response: The API returns data, status codes, or error messages.

API calls let different systems talk to each other by sending requests and receiving structured responses.

Key components of an API call

- **HTTP method:** The type of operation you want to perform (GET, POST, PUT, PATCH, DELETE)
- **Endpoint URL:** The address of the resource you're requesting
- **Headers:** Metadata about the request, including authentication and content type
- **Request body:** The data you're sending with POST, PUT, or PATCH requests
- **Query parameters:** Optional filters appended to the URL.

Types of API calls

A **Request** is when your app asks the server for something. It includes key parts:

URL (the address)

your request needs a URL like:

<https://api.product.com/users/45>

Method (the action)

This tells the server *what you want to do*:

- **GET** → “Give me something”
- **POST** → “Here’s something new”
- **PUT** → “Replace this existing thing”
- **PATCH** → “Update a part of this”
- **DELETE** → “Remove it”

2. Headers (info about the message)

Headers are like labels on a package: who sent it, what’s inside, any access keys, etc. Remember me for faster sign in

3. *Body (optional content)*

Used with POST, PUT, PATCH. The body contains the actual data you're sending, like filling out a form.

A **response** typically includes:

- Status code: Describes outcome (200, 201, 204, 400, 401, 404, 429, 500).
- Headers: Metadata (Content-Type, Cache-Control, rate limit headers).
- Body: Data representation (often JSON).

Example:



Testing s Validation Strategy

Validation testing ensures that the RetailPipe system meets requirements by verifying data accuracy and completeness. Any inconsistencies detected during the ETL process are treated as issues and resolved.

Unit Testing Unit testing is performed to validate individual components of the ETL pipeline. Each function, such as data extraction, transformation, and loading, is tested independently to ensure correct functionality.

For example, transformation logic is tested to verify that null values are handled properly and calculations such as total sales are accurate.

Integration-Testing

Integration testing ensures that all components of the RetailPipe system work together correctly. This includes verifying that data flows seamlessly from data sources through the ETL pipeline and is successfully stored in the data warehouse. The interaction between the database, API layer, and dashboard is also tested to ensure consistency and reliability.

User Acceptance Testing (UAT)

User Acceptance Testing is conducted to ensure that the system meets business requirements and user expectations. End users validate that the dashboard provides meaningful insights such as sales trends and product performance, and that the system outputs are accurate and easy to understand.

Data quality Checklist

Data quality refers to

Accuracy

Sales totals are validated by comparing transformed data with raw transaction data.

Completeness

Missing values in customer and product datasets are handled during the transformation stage.

Consistency

Date formats and currency values are standardized across all datasets

Uniqueness

Duplicate transaction records are removed during preprocessing.

Quality Assurance (QA)

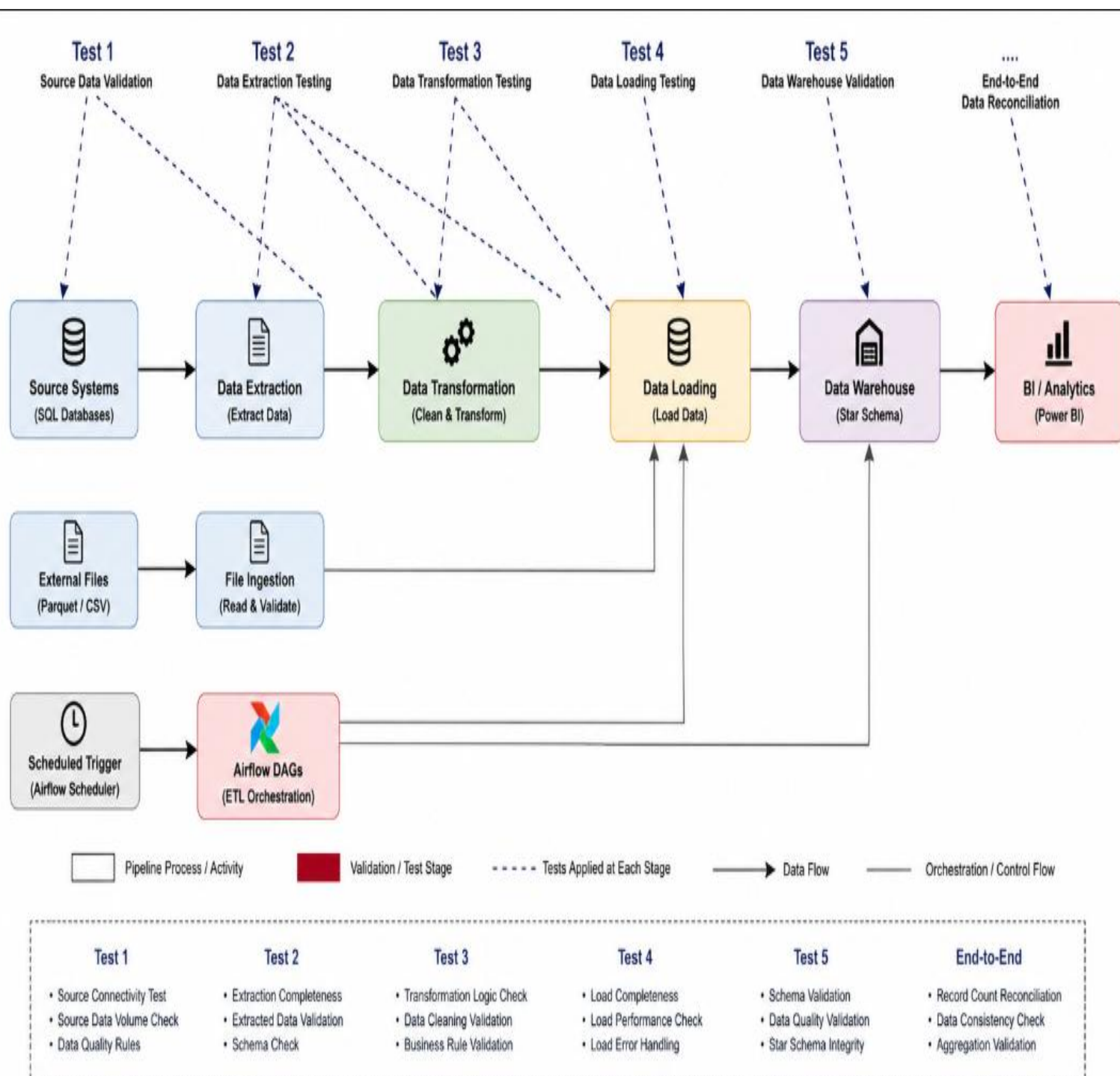
Quality Assurance (QA) is a proactive process focused on preventing errors before they occur within the data pipeline. In the RetailPipe system, QA ensures that all ETL processes are designed with proper validation rules and constraints to maintain data quality from the beginning.

QA is process-oriented and is applied during pipeline design and development. It includes enforcing data validation rules such as restricting invalid values, ensuring correct data types, and defining acceptable ranges for key attributes.

Quality Control (QC)

Quality Control (QC) is a reactive process that focuses on detecting and correcting errors after data has been processed. In RetailPipe, QC is applied after each stage of the ETL pipeline to verify data accuracy and consistency.

QA and QC processes are applied throughout the ETL pipeline to ensure data accuracy at each stage (extraction, transformation, and loading).



Deployment Strategy

The **RetailPipe** system is deployed in a cloud environment to ensure availability, scalability, and easy maintenance. The system includes an automated ETL pipeline that processes retail data from multiple sources and prepares it for analytics., a relational database, RESTful APIs, and an interactive dashboard for business users.

1. Hosting Environment

System components are hosted in a cloud-based environment to ensure reliability and accessibility.

Component	Hosting Environment
ETL Pipeline	Python scripts scheduled on a cloud server (AWS EC2 / Render Cron Jobs)
Database	PostgreSQL (Hosted on Amazon RDS / Render PostgreSQL)
API Layer	Flask REST API hosted on Render (Web Service)
Dashboard	Streamlit application hosted on Streamlit Cloud / Render
File Storage	Raw datasets stored in cloud storage (AWS S3 / Google Drive)

2. Deployment Pipeline (CI/CD)

- A simple CI/CD pipeline is followed to keep the application updated and stable:
- Code Commit: Code is pushed to the GitHub repository.

- **Automated Build:** GitHub Actions runs basic checks and validates the code.
- **Build & Deploy:** The application is built (dependencies installed) and deployed to the cloud server.
- **Deploy:** The API, ETL jobs, and Dashboard are deployed to their respective services.
- **Monitor:** System health and job logs are monitored regularly.

3. *Scaling Considerations*

The RetailPipe system is designed to handle increasing data volume and user load:

- **Data Scaling:**
 - Partitioning large tables by date (e.g., sales fact).
 - Indexing on key columns (date key, product key, store key).
 - Optimized ETL jobs with batch processing.
 - Can migrate to distributed processing (e.g., Apache Spark) for big data.
- **Application Scaling:**
 - The Flask API can be horizontally scaled using multiple instances.
 - A load balancer distributes user requests across instances.
- **Dashboard Scaling:**
 - Caching frequently used queries.
 - Using aggregated tables (daily/monthly summaries) for faster performance.

Future Enhancements:

- Containerization using Docker.
- Orchestration using Kubernetes if traffic increases.
- Real-time data pipeline using streaming tools (e.g., Kafka, Spark Streaming).

Simple Architecture View:

Data Sources (sales) → ETL Pipeline → Data warehouse → API → Dashboard insights

Conclusion:

This section presented the additional deliverables for the RetailPipe system, including API documentation, testing and validation strategy, and deployment approach. These components ensure that the system is not only well-designed at the architectural level, but also reliable, scalable, and ready for real-world implementation. Together, they support seamless data flow, system stability, and efficient communication between all components of the pipeline.
